Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Área Profesional Ingeniería

Introducción a la Programacion y Computacion

“Manual Tecnico Proyecto 1 IPC2 2026 1 semestre”

César Enrique Ixquiac Vásquez./ Carné 202230332

Quetzaltenango, 12 de Marzo 2025

“Id y enseñad a todos”

1. Introducción y Objetivos

Objetivo: Detallar el diseño arquitectónico, la estructura del código en sus capas de Frontend y Backend, la lógica de transacciones relacionales y los mecanismos de seguridad del "Sistema de Gestión de Agencia de Viajes". Este documento sirve como guía para futuros mantenimientos, integración de nuevas APIs y escalabilidad del sistema.

Alcance del sistema: El sistema está diseñado bajo una arquitectura Cliente-Servidor que gestiona tres áreas críticas: Administración Financiera, Operaciones y Atención al Cliente. Implementa un Control de Acceso Basado en Roles (RBAC), persistencia de datos relacional altamente normalizada, generación de reportes dinámicos y un módulo de carga masiva para la migración de datos iniciales.

2. Arquitectura del Sistema y Patrones de Diseño

A diferencia de las arquitecturas monolíticas de escritorio, este proyecto implementa una arquitectura desacoplada:

- **Frontend (Single Page Application):** Desarrollado con el framework **Angular**. Utiliza una arquitectura basada en Componentes y Servicios para consumir recursos remotos de forma asíncrona, brindando una experiencia de usuario fluida sin recargar la página.
- **Backend (API RESTful):** Desarrollado en **Java Enterprise Edition (Servlets)**. Actúa como el motor lógico del sistema.
- **Patrón DAO (Data Access Object):** Separa la lógica de acceso a datos de la lógica de negocio. Los Servlets (Controladores) nunca escriben SQL; delegan esta tarea a las clases DAO.
- **Comunicación HTTP y CORS:** El intercambio de información entre el puerto 4200 (Angular) y el puerto 8080 (Tomcat) se realiza en formato **JSON**. Se implementaron filtros CORS en el backend para autorizar peticiones entre distintos orígenes.

3.1. Estructura Backend (Java)

Organizado modularmente en los siguientes paquetes:

- **Paquete db:** Contiene ConexionDB.java, aplicando el patrón Singleton para mantener un pool de conexiones optimizado hacia MySQL.
- **Paquete model (POJOs):** Clases Planas como Usuario.java, Paquete.java y Reservacion.java que estructuran la información y la preparan para su serialización a JSON mediante la librería Gson.
- **Paquete dao:** * UsuarioDAO.java: Maneja la autenticación y la gestión de roles.
    - ReportesDAO.java: Concentra las consultas analíticas complejas, empleando subconsultas y funciones de agregación para cálculos financieros.
- **Paquete controllers (Servlets):** Punto de entrada de la API. Clases como ReportesServlet.java capturan las peticiones HTTP (GET, POST, PUT), extraen parámetros y retornan respuestas con sus respectivos códigos de estado HTTP (200 OK, 400 Bad Request, etc.).

3.2. Estructura Frontend (Angular)

- **Carpeta core/services:** Contiene clases como admin.service.ts encargadas de inyectar HttpClient para realizar las peticiones a la API en Java.
- **Carpeta features:** Contiene los componentes visuales.
    - reportes.component: Renderiza dinámicamente las métricas. Implementa CSS específico (@media print) para transformar la vista web en un documento PDF imprimible.
    - gestion-paquetes.component: Interfaz compleja que maneja arreglos dinámicos en memoria antes de enviar el "Payload" final al servidor.
- **Carpeta guards:** Archivos de seguridad como auth.guard.ts que interceptan la navegación del usuario y validan su rol almacenado en el navegador, previniendo accesos no autorizados a través de la URL.

4. Lógica de Negocio y Componentes Críticos

4.1. Transacciones Relacionales (Maestro-Detalle) La creación de un Paquete Turístico requiere guardar datos en la tabla principal (paquete) y en una tabla secundaria (servicio_paquete). Para garantizar la integridad de los datos, el Backend desactiva el AutoCommit de MySQL (conn.setAutoCommit(false)). Si ocurre un error al guardar un servicio, se ejecuta un conn.rollback() para deshacer la creación del paquete, evitando datos huérfanos.

4.2. Carga Masiva y Validación El módulo de importación lee estructuras complejas y las procesa en memoria. Se aplican validaciones estrictas en dos niveles:

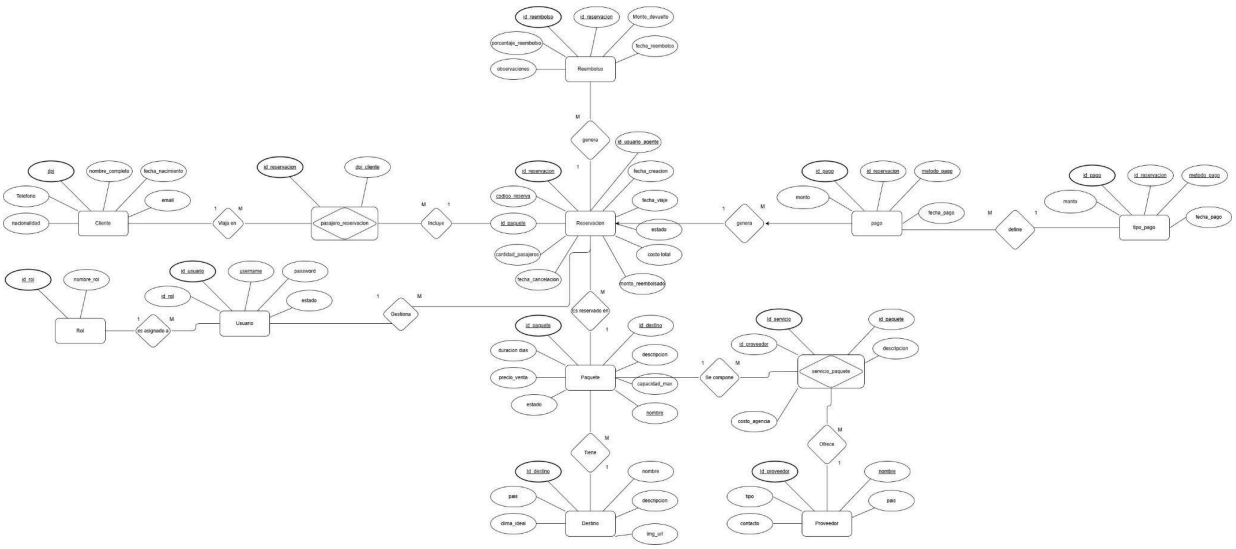
- **Formato:** Expresiones regulares para fechas y validación de tipos numéricos.
- **Lógica:** Verificación de existencia de llaves foráneas (ej. asignar un servicio a un proveedor inexistente) y detección de duplicados (ej. *Usernames* repetidos).

4.3. Motor de Reportes Financieros La generación de métricas (Ganancias, Cancelaciones, Top Agentes) se realiza íntegramente a nivel de base de datos utilizando la cláusula BETWEEN para filtros temporales, LEFT JOIN para incluir reservaciones con reembolsos parciales o nulos, y subconsultas en los bloques SELECT para evitar la duplicación de sumatorias financieras.

5. Modelo Relacional de Base de Datos

La base de datos relacional (agencia_viajes) está altamente normalizada para evitar redundancias y garantizar la consistencia financiera.

- **Tablas Puente:** Se implementaron entidades asociativas como pasajero_reservacion y servicio_paquete para resolver la cardinalidad de Muchos a Muchos (N:M).
- **Area Financiera:** El costo total del viaje (reservacion), los ingresos (pago) y las devoluciones (reembolso) viven en entidades separadas, lo que permite auditorías cruzadas exactas para definir la ganancia neta.



6. Requisitos y Entorno de Despliegue

- **Entorno Backend:**
  1. Lenguaje: Java (JDK 17+)
  2. Servidor de Aplicaciones: Apache Tomcat 10.x.
  3. Base de Datos: MySQL Server 8.0+ (Motor InnoDB obligatorio).
  4. Librerías Externas: mysql-connector-j.jar (JDBC) y gson.jar (Serialización JSON).
- **Entorno Frontend:**
  1. Plataforma: Node.js v18+.
  2. Framework: Angular CLI v17+.
- **Pasos de Despliegue Local:**
  1. Ejecutar el script SQL en el puerto 3306.
  2. Ajustar credenciales en ConexionDB.java.
  3. Desplegar el archivo .war generado por NetBeans en Tomcat (Puerto 8080).
  4. Desde la terminal en la carpeta del frontend, ejecutar npm install seguido de ng serve.
  5. Acceder al sistema en http://localhost:4200.

Mapeo Físico DB

Estructura

```
CREATE DATABASE IF NOT EXISTS agencia_viajes;
USE agencia_viajes;
```

```sql
CREATE TABLE cliente (
  dpi VARCHAR(20) PRIMARY KEY,
  nombre_completo VARCHAR(150) NOT NULL,
  fecha_nacimiento DATE NOT NULL,
  telefono VARCHAR(20),
  email VARCHAR(100),
  nacionalidad VARCHAR(50)
);

CREATE TABLE destino (
  id_destino INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL UNIQUE,
  pais VARCHAR(100) NOT NULL,
  descripcion TEXT,
  clima_epoca_ideal VARCHAR(255),
  imagen_url VARCHAR(500)
);

CREATE TABLE tipo_pago (
  id_tipo_pago INT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  activo TINYINT(1) DEFAULT 1
);

CREATE TABLE proveedor (
  id_proveedor INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL UNIQUE,
  tipo INT NOT NULL,
  pais VARCHAR(100) NOT NULL,
  contacto VARCHAR(255),
  CHECK (tipo IN (1,2,3,4,5))
);

CREATE TABLE paquete (
  id_paquete INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(150) NOT NULL UNIQUE,
  id_destino INT NOT NULL,
  duracion_dias INT NOT NULL,
  descripcion TEXT,
  precio_venta DECIMAL(12,2) NOT NULL,
  capacidad_max INT NOT NULL,
  estado TINYINT(1) DEFAULT 1,
  FOREIGN KEY (id_destino) REFERENCES destino(id_destino)
);
```

```sql
CREATE TABLE rol (
  id_rol INT PRIMARY KEY,
  nombre_rol VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE usuario (
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  id_rol INT NOT NULL,
  estado VARCHAR(20) DEFAULT 'ACTIVO',
  FOREIGN KEY (id_rol) REFERENCES rol(id_rol)
);

CREATE TABLE reservacion (
  id_reservacion INT AUTO_INCREMENT PRIMARY KEY,
  codigo_reserva VARCHAR(20) UNIQUE,
  id_paquete INT NOT NULL,
  id_usuario_agente INT NOT NULL,
  fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,
  fecha_viaje DATE NOT NULL,
  estado ENUM('Pendiente','Confirmada','Cancelada','Completada') DEFAULT
'Pendiente',
  costo_total DECIMAL(12,2) NOT NULL,
  cantidad_pasajeros INT DEFAULT 1,
  fecha_cancelacion DATETIME,
  monto_reembolsado DECIMAL(12,2) DEFAULT 0.00,
  FOREIGN KEY (id_paquete) REFERENCES paquete(id_paquete),
  FOREIGN KEY (id_usuario_agente) REFERENCES usuario(id_usuario)
);

CREATE TABLE pago (
  id_pago INT AUTO_INCREMENT PRIMARY KEY,
  id_reservacion INT NOT NULL,
  monto DECIMAL(12,2) NOT NULL,
  metodo_pago INT NOT NULL,
  fecha_pago DATE NOT NULL,
  FOREIGN KEY (id_reservacion) REFERENCES reservacion(id_reservacion),
  FOREIGN KEY (metodo_pago) REFERENCES tipo_pago(id_tipo_pago),
  CHECK (metodo_pago IN (1,2,3))
);

CREATE TABLE pasajero_reservacion (
  id_reservacion INT,
  dpi_cliente VARCHAR(20),
  PRIMARY KEY (id_reservacion, dpi_cliente),
```

```sql
  FOREIGN KEY (id_reservacion) REFERENCES reservacion(id_reservacion),
  FOREIGN KEY (dpi_cliente) REFERENCES cliente(dpi)
);

CREATE TABLE servicio_paquete (
  id_servicio INT AUTO_INCREMENT PRIMARY KEY,
  id_paquete INT NOT NULL,
  id_proveedor INT NOT NULL,
  descripcion VARCHAR(255),
  costo_agencia DECIMAL(12,2) NOT NULL,
  FOREIGN KEY (id_paquete) REFERENCES paquete(id_paquete),
  FOREIGN KEY (id_proveedor) REFERENCES proveedor(id_proveedor)
);

CREATE TABLE reembolso (
  id_reembolso INT AUTO_INCREMENT PRIMARY KEY,
  id_reservacion INT NOT NULL,
  monto_devuelto DECIMAL(10,2) NOT NULL,
  porcentaje_reembolso INT NOT NULL,
  fecha_reembolso DATE NOT NULL,
  observaciones VARCHAR(255),
  FOREIGN KEY (id_reservacion) REFERENCES reservacion(id_reservacion)
);
```

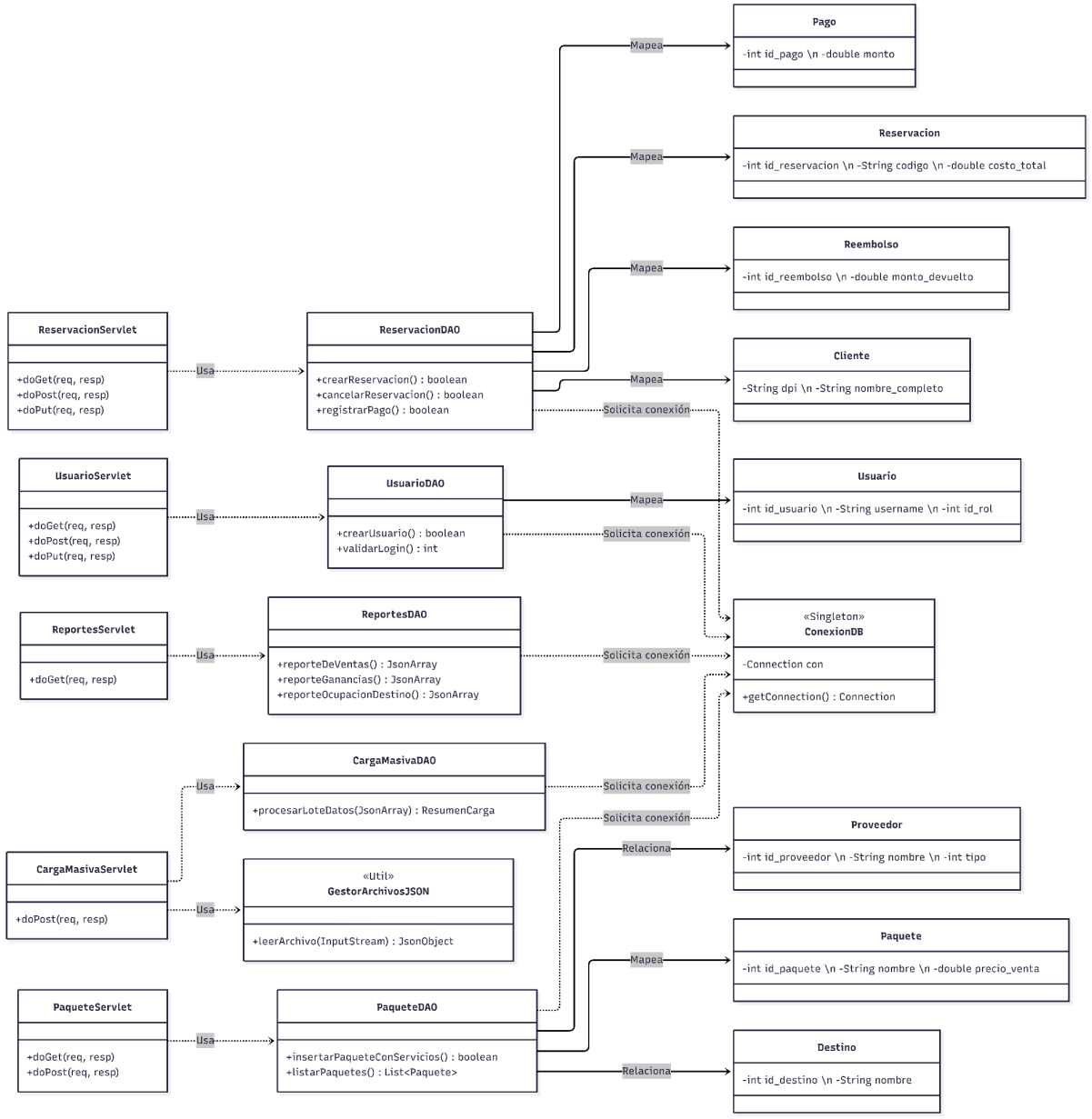
Datos de pruebas

```sql
INSERT INTO cliente VALUES
('1234567890101', 'Maria Lopez', '1990-03-15', '55551234',
'mlopez@mail.com', 'Guatemalteca'),
('2345678901011', 'Juan Perez', '1985-05-20', '44445555',
'juanp@mail.com', 'Guatemalteca'),
('3132159950901', 'Cesar Enrique Ixquiac Vasquez', '2004-08-31',
'58546762', 'cesarixquiac49@gmail.com', 'Guatemalteco'),
('5839201940101', 'Paco perez', '2026-04-16', '58546762',
'cesarixquiac49@gmail.com', 'Guatemalteco');

INSERT INTO destino VALUES
(1, 'Quetzaltenango', 'Guatemala', 'La cuna de la cultura, ideal para
senderismo y volcanes.', NULL, NULL),
(2, 'Cancún', 'México', 'Playas de arena blanca y clima tropical.', NULL,
'https://www.everysteph.com/wp-content/uploads/2019/08/things-to-do-in-can
cun-featured-2-1-1920x1000.jpg'),
(3, 'Antigua Guatemala', 'Guatemala', 'Ciudad colonial con mucha
historia.', NULL, NULL),
(4, 'Peten', 'Guatemala', 'Cuna de la civilizacion Maya, hogar de Tikal',
```

```sql
NULL, NULL),
(7, 'Machupichu', 'Peru', 'Ruinas e historia', 'Mejor epoca Otoño calido',
NULL),
(8, 'Madrid', 'Europa', 'Visita las calles de madrid', 'Mejor epoca para
visitar otoño', NULL);

INSERT INTO tipo_pago VALUES
(1, 'Efectivo', 1),
(2, 'Tarjeta de Crédito/Débito', 1),
(3, 'Transferencia Bancaria', 1);

INSERT INTO rol VALUES
(3, 'Administrador'),
(1, 'Atencion al Cliente'),
(2, 'Operaciones');

INSERT INTO usuario VALUES
(1, 'admin_cesar', 'admin123', 3, 'ACTIVO'),
(2, 'jperez', '123', 3, 'INACTIVO'),
(3, 'operador_xela', '123', 2, 'ACTIVO'),
(4, 'mlopez', 'Segura123', 1, 'ACTIVO'),
(5, 'admin_jefe', 'SuperPass', 3, 'ACTIVO'),
(6, 'nuevo_admin', 'secreta123', 3, 'ACTIVO'),
(7, 'agente_xela', 'viajes2026', 1, 'ACTIVO'),
(8, 'CalosLopez', 'admin123', 3, 'ACTIVO');

INSERT INTO proveedor VALUES
(1, 'TACA Airlines', 1, 'Guatemala', NULL),
(2, 'Hotel Decameron', 2, 'México', NULL),
(3, 'Transportes Xela-Bus', 4, 'Guatemala', NULL),
(4, 'Avianca', 1, 'Colombia', NULL),
(5, 'Transportes Galgos', 4, 'Guatemala', NULL),
(6, 'Hotel Barcelo', 2, 'Guatemala', 'barcelo@hotel.com');
```

Estructura del Diagrama de Clases (Backend)

El presente diagrama expone la arquitectura completa del API REST desarrollado en Java. Demuestra la estricta adherencia al **Principio de Responsabilidad Única (SRP)**:

- La capa superior (Controladores) se limita exclusivamente a interceptar el tráfico HTTP y rutearlo.
- La capa media (DAOs) concentra toda la lógica transaccional de MySQL.
- La capa inferior (POJOs) representa en memoria la estructura de la base de datos relacional.

Se omiten deliberadamente los métodos accesores (getters/setters) de los Modelos en la representación visual para preservar la legibilidad general de la arquitectura.